MODUL 6

GANGGUAN DAN KUALITAS TRANSMISI

Memahami Attenuation, Distortion, Noise, SNR, Bit Error, dan Bit Error Rate melalui Python

Mata Kuliah : Komunikasi Data
Program Studi : Teknik Informatika
Semester : 3
Bobot : 2 SKS
Model Pembelajaran : Concept Exploration, Guided Coding, Simulation, Case Study, dan Project-Based Learning
Perangkat Praktikum : Python melalui Google Colab/Jupyter Notebook

1. Deskripsi dan Capaian Pembelajaran

Pada Modul 5 kita telah mempelajari bagaimana data digital direpresentasikan menjadi sinyal digital menggunakan berbagai teknik line coding. Sender dapat menentukan pola sinyal tertentu untuk membawa bit 0 dan 1 menuju receiver.

Namun, muncul persoalan berikutnya:

Apakah sinyal yang tiba di receiver selalu sama persis dengan sinyal yang dikirim sender?

Jawabannya tidak selalu.

Selama melewati transmission medium, sinyal dapat mengalami perubahan. Kekuatan sinyal dapat berkurang, bentuk sinyal dapat berubah, dan gangguan dari lingkungan dapat bercampur dengan sinyal yang sebenarnya ingin diterima. Kondisi tersebut secara umum disebut **transmission impairment**.

Pada komunikasi analog, gangguan dapat menyebabkan penurunan kualitas sinyal. Pada komunikasi digital, gangguan yang cukup besar bahkan dapat menyebabkan receiver salah menafsirkan nilai bit.

Sender mungkin mengirim:

10110110

tetapi receiver memperoleh:

10100110

Hanya satu bit yang berubah, tetapi perubahan tersebut dapat memengaruhi data yang diterima.

Karena itu sistem komunikasi tidak hanya perlu mampu mengirim data. Engineer juga perlu mengetahui **seberapa baik kualitas transmisi** dan **seberapa sering kesalahan terjadi**.

Pada modul ini mahasiswa akan mempelajari tiga bentuk gangguan transmisi utama, yaitu **attenuation, distortion**, dan **noise**. Pembahasan dilanjutkan dengan konsep **Signal-to-Noise Ratio (SNR), bit error**, dan **Bit Error Rate (BER)**.

Python digunakan untuk membantu mahasiswa melihat pengaruh gangguan secara visual dan melakukan simulasi perubahan bit secara acak.

Setelah menyelesaikan modul ini mahasiswa diharapkan mampu:

1. menjelaskan konsep transmission impairment;
2. membedakan attenuation, distortion, dan noise;
3. menjelaskan hubungan sinyal dan noise melalui SNR;
4. menghitung SNR sederhana dalam bentuk rasio dan decibel;
5. menjelaskan bagaimana gangguan dapat menghasilkan bit error;
6. menghitung Bit Error Rate;
7. mensimulasikan noise dan bit error menggunakan Python;
8. menganalisis hubungan probabilitas error dengan BER; dan
9. menambahkan fitur **Channel Impairment Simulator** pada Data Communication Simulator.

2. Mengapa Sinyal Dapat Berubah Selama Transmisi?

Pada kondisi ideal, receiver menerima sinyal yang identik dengan sinyal yang dikirim oleh sender.

Namun transmission medium bukanlah jalur yang sempurna. Ketika sinyal merambat melalui kabel, fiber optic, maupun ruang bebas, karakteristik fisiknya dapat berubah.

Sebagian energi sinyal dapat berkurang selama perjalanan. Berbagai komponen frekuensi dapat mengalami perubahan yang tidak sama. Selain itu, terdapat kemungkinan sinyal lain atau gangguan fisik ikut memengaruhi sinyal utama.

Akibatnya, sinyal yang tiba di receiver dapat berbeda dari sinyal yang dikirim.

Dalam komunikasi data, perubahan tersebut penting karena receiver harus mengambil keputusan berdasarkan sinyal yang diterimanya.

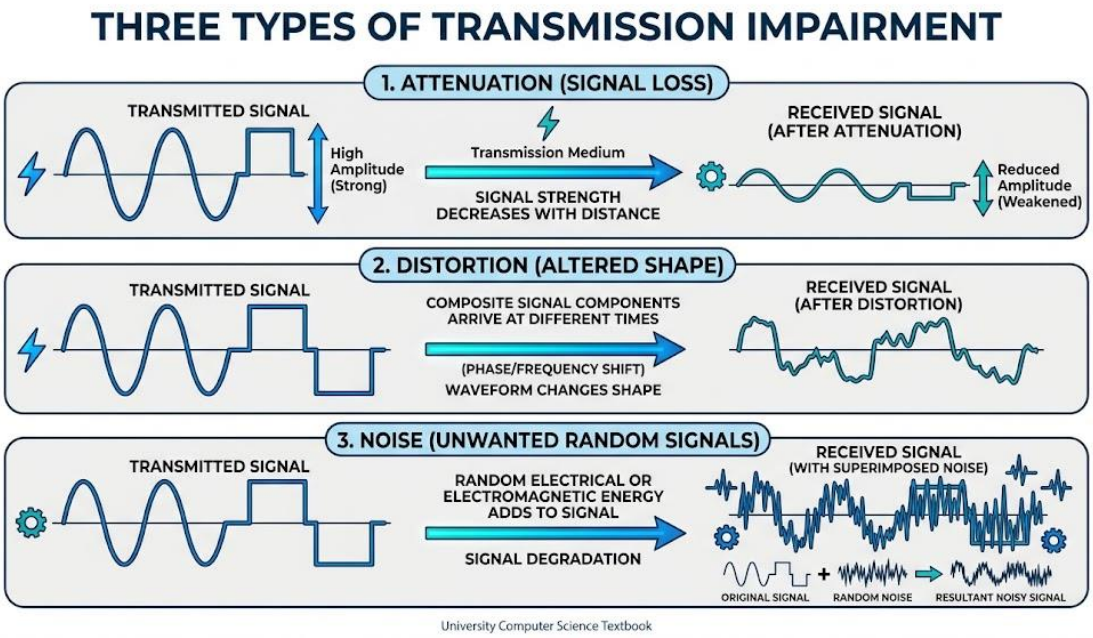
Misalnya receiver menggunakan level tertentu untuk membedakan bit 0 dan 1. Jika gangguan membuat level sinyal bergeser cukup jauh, receiver dapat mengambil keputusan yang salah.

Masalah pada proses transmisi umumnya dapat dikelompokkan menjadi:

- **attenuation**;
- **distortion**;
- **noise**.

Ketiganya mempunyai karakteristik yang berbeda, tetapi semuanya dapat menurunkan kualitas komunikasi.

Referensi teori: Forouzan (2013), bagian Transmission Impairment; Stallings (2022), bagian Transmission Impairments.



Gambar 6.1 — Transmission Impairment

3. Attenuation — Ketika Sinyal Melemah

Attenuation adalah berkurangnya kekuatan sinyal ketika sinyal merambat melalui media transmisi.

Bayangkan suara seseorang yang terdengar semakin lemah ketika jarak antara pembicara dan pendengar semakin jauh. Konsep yang sama secara umum dapat digunakan untuk membayangkan attenuation, walaupun mekanisme fisiknya pada sistem komunikasi tentu berbeda.

Pada kabel, fiber optic, dan komunikasi wireless, sinyal dapat kehilangan sebagian energinya sepanjang jalur transmisi.

Jika sinyal menjadi terlalu lemah ketika mencapai receiver, perangkat penerima dapat mengalami kesulitan membedakan sinyal yang sebenarnya dari noise.

Untuk mengatasi attenuation pada sistem nyata dapat digunakan berbagai teknik, misalnya amplifier atau repeater, tergantung teknologi komunikasi yang digunakan.

Attenuation sering dinyatakan dalam **decibel (dB)**. Untuk membandingkan daya input dan output dapat digunakan:

$$dB = 10 \log_{10} \left(\frac{P_{out}}{P_{in}} \right)$$

Jika daya output lebih kecil dari daya input, hasil perhitungan tersebut bernilai negatif, yang menunjukkan terjadinya kehilangan daya.

Pada modul ini mahasiswa tidak perlu mendalami link budget. Tujuannya adalah memahami bahwa **semakin jauh atau semakin tidak ideal media yang dilewati, kekuatan sinyal dapat semakin berkurang**.

4. Distortion — Ketika Bentuk Sinyal Berubah

Distortion terjadi ketika bentuk sinyal yang diterima tidak lagi sama dengan bentuk sinyal yang dikirim.

Sebuah sinyal kompleks dapat terdiri atas beberapa komponen frekuensi. Media transmisi tidak selalu memberikan respons yang sama terhadap setiap komponen tersebut. Sebagian komponen dapat mengalami attenuation atau delay yang berbeda.

Akibatnya, ketika seluruh komponen kembali bergabung di receiver, bentuk keseluruhan sinyal dapat berubah.

Distortion berbeda dengan attenuation sederhana.

Pada attenuation, bentuk dasar sinyal dapat tetap serupa tetapi amplitudonya mengecil.

Pada distortion, **bentuk sinyal itu sendiri berubah**.

Perubahan bentuk tersebut dapat menjadi masalah pada komunikasi digital karena receiver harus menentukan posisi transisi dan level sinyal dengan benar.

Semakin besar distortion, semakin besar kemungkinan receiver salah menginterpretasikan sinyal.

Referensi teori: Forouzan (2013); Stallings (2022).

5. Noise — Gangguan yang Ikut Masuk ke Sinyal

Noise merupakan energi atau sinyal yang tidak diinginkan dan ikut memengaruhi sinyal utama.

Dalam dunia nyata terdapat berbagai sumber noise. Forouzan membahas beberapa bentuk seperti thermal noise, induced noise, crosstalk, dan impulse noise.

Pada tingkat pengantar, mahasiswa tidak perlu menghafalkan seluruh karakteristik fisiknya. Yang lebih penting adalah memahami bahwa noise dapat berasal dari sistem elektronik, lingkungan, perangkat lain, maupun fenomena fisik tertentu.

Ketika noise bercampur dengan sinyal, receiver memperoleh gabungan antara:

sinyal yang diinginkan

dan

gangguan yang tidak diinginkan.

Jika sinyal jauh lebih kuat daripada noise, receiver relatif lebih mudah membedakan informasi yang sebenarnya.

Jika noise menjadi besar dibandingkan sinyal, proses deteksi menjadi semakin sulit.

Kondisi inilah yang membawa kita pada konsep **Signal-to-Noise Ratio**.

6. Praktikum Python 1 — Melihat Pengaruh Noise

Kita akan membuat sinyal sinus sederhana kemudian menambahkan noise acak.

Kode Program 6.1

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 1000)

frekuensi = 5
amplitudo = 1

sinyal = amplitudo * np.sin(
    2 * np.pi * frekuensi * t
)

noise = np.random.normal(
    0,
    0.3,
    len(t)
)

sinyal_noisy = sinyal + noise

plt.plot(t, sinyal)
plt.plot(t, sinyal_noisy)

plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.title("Sinyal Asli dan Sinyal dengan Noise")

plt.grid()
plt.show()
```

Memahami Command

Command	Penjelasan
np.linspace(0, 1, 1000)	Membuat 1000 titik waktu dari 0 sampai 1 detik.
np.sin()	Menghasilkan gelombang sinus.
np.random.normal()	Menghasilkan nilai acak dengan distribusi normal.
0	Nilai rata-rata noise pada simulasi.
0.3	Standar deviasi yang menentukan besar variasi noise.
len(t)	Menentukan jumlah nilai noise sama dengan jumlah sampel waktu.
sinyal + noise	Menambahkan gangguan pada sinyal asli.
plt.plot()	Menggambar data sebagai grafik.

Apa yang Terjadi?

Program mempunyai dua data:

```
sinyal
```

dan:

```
sinyal_noisy
```

Sinyal pertama merupakan bentuk ideal.

Sinyal kedua merupakan hasil setelah ditambahkan nilai acak.

Perhatikan bahwa noise tidak harus menghilangkan seluruh bentuk sinyal. Pada tingkat tertentu kita masih dapat melihat bentuk sinyal asli, tetapi garisnya menjadi lebih tidak teratur.

Coba Otak-Atik

Ubah:



menjadi:



kemudian:



dan:



Amati perubahan grafik.

Pertanyaan

1. Apa yang terjadi ketika nilai standar deviasi noise diperbesar?
2. Apakah bentuk sinyal asli semakin mudah atau semakin sulit dikenali?
3. Mengapa noise yang besar dapat meningkatkan kemungkinan kesalahan pada receiver?

7. Signal-to-Noise Ratio

Untuk menggambarkan hubungan antara kekuatan sinyal dan noise digunakan **Signal-to-Noise Ratio (SNR)**.

Secara sederhana:

$$SNR = \frac{P_{signal}}{P_{noise}}$$

Jika daya sinyal jauh lebih besar daripada daya noise, nilai SNR tinggi.

Jika daya sinyal hampir sama atau bahkan lebih kecil daripada noise, nilai SNR rendah.

SNR sering dinyatakan dalam decibel:

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right)$$

Sebagai contoh, jika:

$$P_{signal} = 100$$

dan:

$$P_{noise} = 1$$

maka:

$$SNR = 100$$

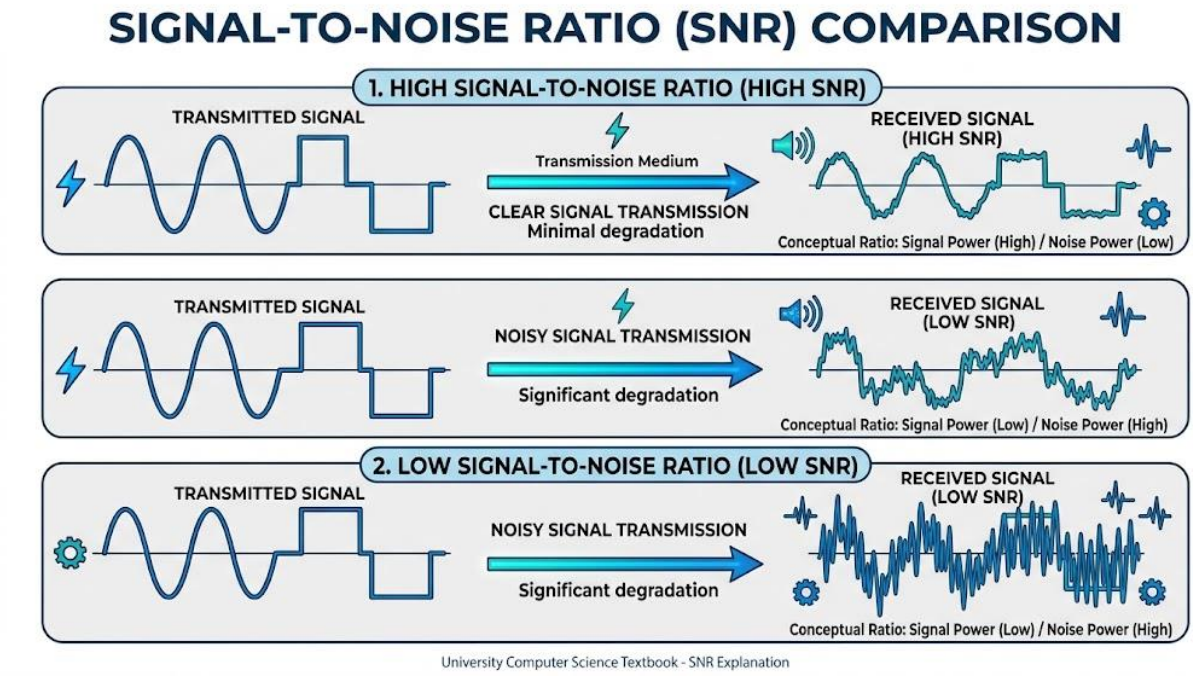
serta:

$$SNR_{dB} = 10log_{10}(100) = 20\text{dB}$$

SNR yang lebih tinggi secara umum menunjukkan bahwa sinyal lebih dominan dibandingkan noise.

Namun SNR bukan satu-satunya faktor yang menentukan kualitas sistem. Teknik modulasi, coding, receiver, bandwidth, dan berbagai kondisi lainnya juga berpengaruh.

Referensi teori: Forouzan (2013), bagian Signal-to-Noise Ratio; Stallings (2022).



Gambar 6.2 — Memahami SNR

8. Praktikum Python 2 — Menghitung SNR

Kode Program 6.2

```
import numpy as np

signal_power = 100
noise_power = 1

snr = signal_power / noise_power

snr_db = 10 * np.log10(snr)

print("Signal Power :", signal_power)
print("Noise Power  :", noise_power)
print("SNR          :", snr)
print("SNR (dB)     :", snr_db)
```

Output

```
Signal Power : 100
Noise Power  : 1
SNR          : 100.0
SNR (dB)     : 20.0
```

Memahami Command

Command	**Penjelasan**
signal_power = 100	Menyimpan nilai daya sinyal.
noise_power = 1	Menyimpan nilai daya noise.
/	Menghitung rasio antara signal power dan noise power.
np.log10()	Menghitung logaritma basis 10.

Command	Penjelasan
`10 * np.log10(snr)`	Mengubah nilai rasio SNR menjadi decibel.

Coba Ubah

Pertahankan:

```
signal_power = 100
```

Kemudian ubah:

```
noise_power
```

menjadi:

`1`

`5`

`10`

`50`

Catat hasilnya.

Signal Power	Noise Power	SNR	SNR (dB)
100	1		

Signal Power	Noise Power	SNR	SNR (dB)
100	5		
100	10		
100	50		

Analisis

1. Apa yang terjadi terhadap SNR ketika noise meningkat?
2. Apakah SNR meningkat atau menurun?
3. Kondisi mana yang secara umum lebih mudah bagi receiver untuk mendeteksi sinyal?

9. Dari Gangguan Sinyal Menjadi Bit Error

Pada komunikasi digital, receiver harus mengambil keputusan berdasarkan sinyal yang diterima.

Misalnya sistem sederhana menggunakan dua level.

Receiver dapat menentukan:

- level tertentu dianggap bit `1`;
- level lainnya dianggap bit `0`.

Jika sinyal mengalami attenuation, distortion, atau noise yang cukup besar, level yang diterima dapat bergeser melewati batas keputusan receiver.

Sender sebenarnya mengirim bit:

`1`

tetapi receiver memutuskan bahwa sinyal tersebut adalah:

`0`.

Kondisi tersebut disebut **bit error**.

Contoh:

Data yang dikirim:

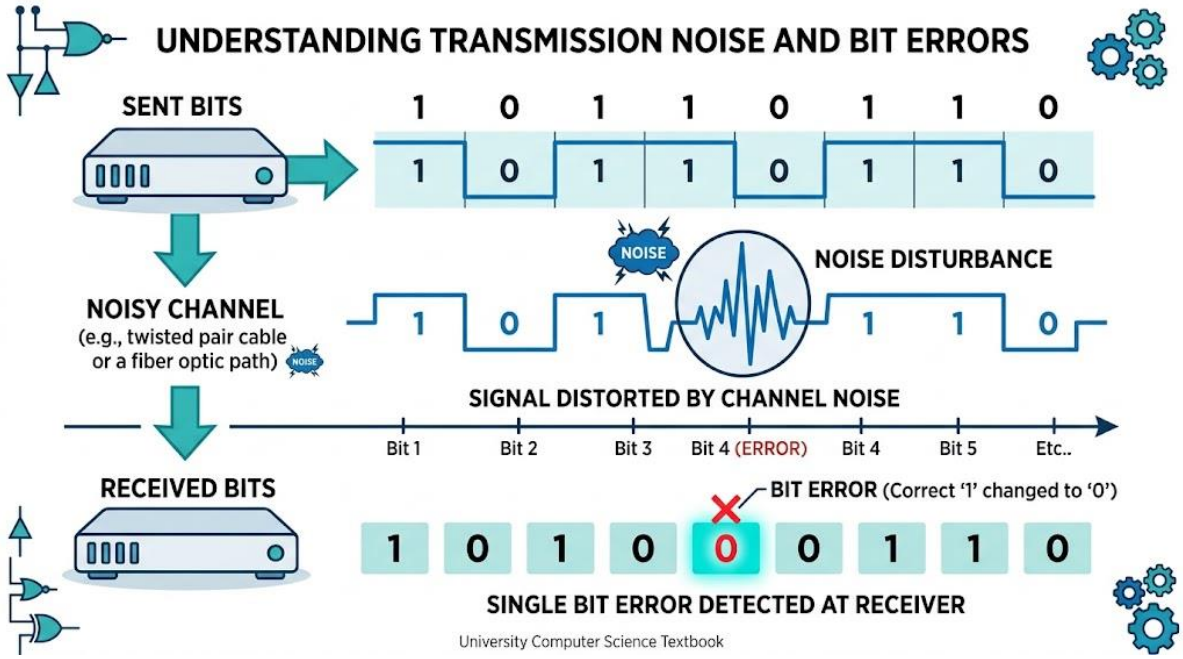
`10110110`

Data yang diterima:

10100110

Perhatikan bahwa terdapat satu posisi dengan nilai berbeda.

Bit error menjadi salah satu indikator penting kualitas komunikasi digital.



Gambar 6.3 — Dari Noise Menjadi Bit Error

10. Bit Error Rate

Mengetahui bahwa terdapat error belum cukup untuk mengevaluasi kualitas sebuah komunikasi.

Kita juga perlu mengetahui seberapa sering error terjadi dibandingkan dengan jumlah bit yang dikirim.

Ukuran tersebut disebut **Bit Error Rate (BER)**.

Secara sederhana:

$$BER = \frac{\text{Jumlah bit yang salah}}{\text{Jumlah seluruh bit yang dikirim}}$$

Misalnya sender mengirim:

$$10.000\text{bit}$$

dan receiver menemukan:

$$50\text{bit error}$$

maka:

$$BER = \frac{50}{10.000} = 0{,}005$$

atau:

$$0{,}5\%$$

Artinya sekitar 0,5% dari seluruh bit yang dikirim mengalami kesalahan.

Semakin kecil BER, semakin sedikit kesalahan yang terjadi pada transmisi.

Namun nilai BER yang dapat diterima bergantung pada karakteristik sistem dan mekanisme komunikasi yang digunakan.

Referensi teori: Forouzan (2013); Stallings (2022).

11. Praktikum Python 3 — Mensimulasikan Bit Error dan BER

Sekarang kita akan mensimulasikan sebuah kanal sederhana yang secara acak dapat membalik nilai bit.

Kode Program 6.3

```
import random

sent_bits = "1011010010110101"

error_probability = 0.10

received_bits = ""
error_count = 0

for bit in sent_bits:

    if random.random() < error_probability:

        if bit == "1":
            received_bits += "0"
        else:
            received_bits += "1"

        error_count += 1

    else:
        received_bits += bit

ber = error_count / len(sent_bits)

print("Sent Bits       :", sent_bits)
print("Received Bits  :", received_bits)
print("Total Bits      :", len(sent_bits))
print("Bit Error       :", error_count)
print("BER             :", ber)
print("BER (%)         :", ber * 100)
```

Memahami Command

Command	Penjelasan
`import random`	Memanggil modul bawaan Python untuk bilangan acak.
`random.random()`	Menghasilkan angka acak antara 0 dan kurang dari 1.
`error_probability = 0.10`	Menentukan probabilitas error sebesar 10% pada model simulasi.
`<`	Operator perbandingan “lebih kecil dari”.
`received_bits = ""`	Membuat string kosong untuk menampung data receiver.
`+=`	Menambahkan nilai pada variabel yang sudah ada.
`error_count += 1`	Menambah penghitung error sebanyak satu.
`len(sent_bits)`	Menghitung jumlah bit yang dikirim.
`error_count / len(sent_bits)`	Menghitung BER.

Bagaimana Simulasi Bekerja?

Untuk setiap bit, Python membuat sebuah angka acak.

Jika angka tersebut lebih kecil dari:

```
error_probability
```

bit dibalik.

Jika bit awal:

`1`

maka menjadi:

0.

Sebaliknya:

0

menjadi:

1.

Perlu dipahami bahwa program ini merupakan **model probabilistik sederhana**. Nilai error_probability bukan model lengkap suatu kanal fisik.

12. Coba Otak-Atik BER

Gunakan data yang lebih panjang agar hasil eksperimen lebih mudah diamati.

Misalnya:

```
sent_bits = "1011010010" * 100
```

Command Baru

Operator:

```
*
```

pada string dapat digunakan untuk mengulang string.

Jika:

```
"10" * 3
```

hasilnya:

```
101010
```

Sekarang lakukan percobaan menggunakan:

```
error_probability = 0.00
```

kemudian:

```
0.01
0.05
0.10
0.20
```

Catat hasilnya.

Error Probability	**Total Bit**	**Bit Error**	**BER**
0.00			
0.01			
0.05			
0.10			
0.20			

Pertanyaan Analisis

1. Apa yang terjadi terhadap BER ketika error probability meningkat?
2. Apakah BER selalu tepat sama dengan error probability?
3. Mengapa hasil simulasi dapat sedikit berbeda setiap kali program dijalankan?
4. Apa yang terjadi jika jumlah bit yang diuji diperbesar?

13. Membandingkan Transmission Impairment

Ketiga jenis impairment yang dipelajari mempunyai karakteristik berbeda.

Gangguan	**Dampak Utama**	**Gambaran Sederhana**
Attenuation	Kekuatan sinyal berkurang	Sinyal semakin lemah
Distortion	Bentuk sinyal berubah	Bentuk receiver berbeda dari sender
Noise	Gangguan tambahan bercampur dengan sinyal	Sinyal menjadi tidak bersih

Gangguan tersebut tidak selalu muncul secara terpisah.

Dalam sistem nyata dapat terjadi beberapa impairment sekaligus. Sinyal dapat mengalami attenuation sekaligus terkena noise dan distortion.

Jika perubahan yang terjadi cukup besar, receiver dapat mengambil keputusan bit yang salah.

Dengan demikian terdapat hubungan konseptual antara:

kualitas sinyal, **SNR**, dan **bit error**.

Namun hubungan matematis secara lengkap bergantung pada banyak faktor dan berada di luar cakupan modul pengantar ini.

14. Studi Kasus — Sensor Wireless di Lingkungan Industri

Sebuah sistem monitoring menggunakan sensor wireless untuk mengirimkan data dari mesin produksi menuju server.

Ketika hanya sebagian mesin beroperasi, hasil pengujian menunjukkan:

- total data = 100.000 bit;
- bit error = 20 bit.

Ketika seluruh mesin beroperasi, hasil pengujian menunjukkan:

- total data = 100.000 bit;
- bit error = 750 bit.

Kondisi Pertama

$$BER = \frac{20}{100.000} = 0{,}0002$$

atau:

$$0{,}02\%$$

Kondisi Kedua

$$BER = \frac{750}{100.000} = 0{,}0075$$

atau:

0,75%

Hasil tersebut menunjukkan peningkatan tingkat kesalahan ketika kondisi lingkungan berubah.

Namun kita tidak boleh langsung menyimpulkan penyebab pasti hanya berdasarkan data BER. Dibutuhkan pemeriksaan lebih lanjut terhadap sumber interferensi, kualitas sinyal, posisi perangkat, media komunikasi, maupun kondisi perangkat.

Tugas Praktikum

Buat program berikut.

```
total_bits = 100000

error_normal = 20
error_industri = 750

ber_normal = error_normal / total_bits
ber_industri = error_industri / total_bits

print("Kondisi Normal")
print("BER :", ber_normal)
print("BER (%):", ber_normal * 100)

print()

print("Seluruh Mesin Beroperasi")
print("BER :", ber_industri)
print("BER (%):", ber_industri * 100)
```

Pertanyaan Analisis

1. Kondisi mana yang menghasilkan BER lebih besar?
2. Berapa kali BER meningkat?
3. Apakah BER saja cukup untuk menentukan sumber masalah?
4. Pengukuran apa lagi yang dapat membantu analisis, misalnya kekuatan sinyal atau SNR?
5. Mengapa lingkungan industri berpotensi memengaruhi komunikasi wireless?

15. Project Semester — Milestone 6

Pada Modul 1 simulator mempunyai **Communication Profile**.

Modul 2 menambahkan **Message Encoder**.

Modul 3 menambahkan **Signal Visualizer**.

Modul 4 menambahkan **Transmission Channel Analyzer**.

Modul 5 menambahkan **Line Coding Visualizer**.

Pada Modul 6 ditambahkan:

Channel Impairment Simulator

Fitur ini mensimulasikan kemungkinan perubahan bit selama transmisi.

Tahap Awal

Buat fungsi:

```
import random

def transmit_with_error(bits, error_probability):

    received = ""
    error_count = 0

    for bit in bits:

        if random.random() < error_probability:

            if bit == "1":
```

```
                received += "0"
            else:
                received += "1"

            error_count += 1

        else:
            received += bit

    return received, error_count
```

Memahami `return` Lebih Lanjut

Pada Modul 5 fungsi menghasilkan satu hasil.

Pada fungsi ini:

```
return received, error_count
```

Python mengembalikan **dua nilai** sekaligus.

Fungsi dapat digunakan:

```
sent_bits = "1011010010"

received_bits, errors = transmit_with_error(
    sent_bits,
    0.10
)
```

Variabel:

```
received_bits
```

menyimpan data hasil transmisi.

Sedangkan:

```
errors
```

menyimpan jumlah bit error.

Menghitung BER

Tambahkan:

```
ber = errors / len(sent_bits)
```

Kemudian tampilkan:

```
print("Sent     :", sent_bits)
print("Received :", received_bits)
print("Errors   :", errors)
print("BER      :", ber)
```

Challenge Project

Program meminta pengguna memasukkan:

```
Binary Data
Error Probability
```

Kemudian menampilkan:

- sent bits;
- received bits;
- total bits;
- number of errors;
- BER;
- BER dalam persen.

Advanced Challenge

Tambahkan fungsi yang menunjukkan **posisi bit yang mengalami error**.

Contoh output:

```
Error ditemukan pada posisi:
3
8
12
```

Petunjuk: mahasiswa dapat mempelajari fungsi:

```
enumerate()
```

untuk memperoleh nilai sekaligus posisi suatu elemen ketika melakukan perulangan.

16. Ringkasan

Transmission medium tidak selalu membawa sinyal dalam kondisi sempurna.

Gangguan utama yang diperkenalkan pada modul ini adalah **attenuation, distortion**, dan **noise**.

Attenuation menyebabkan kekuatan sinyal berkurang selama transmisi.

Distortion menyebabkan bentuk sinyal berubah.

Noise merupakan gangguan yang tidak diinginkan dan bercampur dengan sinyal utama.

Hubungan antara kekuatan sinyal dan noise dapat digambarkan menggunakan **Signal-to-Noise Ratio (SNR)**:

$$SNR = \frac{P_{signal}}{P_{noise}}$$

Dalam decibel:

$$SNR_{dB} = 10log_{10}(SNR)$$

Jika gangguan menyebabkan receiver salah menentukan nilai bit, terjadi **bit error**.

Tingkat kesalahan dapat dihitung menggunakan:

$$BER = \frac{\text{Bit Error}}{\text{Total Bit}}$$

Pada modul ini mahasiswa menggunakan Python untuk menambahkan noise, menghitung SNR, mensimulasikan bit error, dan menghitung BER.

Kemampuan Python yang diperkuat meliputi:

np.random.normal(), np.log10(), random.random(), for, if, string concatenation, +=, function, dan multiple return values.

Pada semester project ditambahkan fitur **Channel Impairment Simulator**.

17. Tes Pemahaman

A. Konseptual

1. Apa yang dimaksud transmission impairment?
2. Jelaskan perbedaan attenuation dan distortion.
3. Apa yang dimaksud noise?
4. Mengapa noise dapat memengaruhi proses deteksi bit?
5. Apa yang dimaksud SNR?
6. Apa arti kondisi SNR yang tinggi secara umum?
7. Apa yang dimaksud bit error?
8. Apa yang dimaksud Bit Error Rate?
9. Mengapa nilai BER yang kecil lebih diinginkan?

B. Perhitungan

1. Sebanyak 10.000 bit dikirim dan terdapat 25 bit error. Hitung BER.
2. Sebanyak 50.000 bit dikirim dan terdapat 500 bit error. Hitung BER dalam persen.
3. Jika signal power = 100 dan noise power = 10, hitung SNR.
4. Hitung SNR dalam dB untuk rasio SNR = 10.

C. Python

Perhatikan:

```
import random

error_probability = 0.05

if random.random() < error_probability:
    print("Error")
else:
    print("Normal")
```

Jawab:

1. Apa fungsi `random.random()`?
2. Apa arti `0.05` pada program?
3. Mengapa output dapat berbeda pada setiap eksekusi?
4. Apakah `0.05` berarti selalu tepat 5 dari setiap 100 bit pasti error? Jelaskan.

18. Tugas Mandiri

Kasus: Evaluasi Kanal Komunikasi Sensor

Sebuah sensor mengirimkan 2.000 bit data pada setiap percobaan.

Buat program Python yang mensimulasikan transmisi menggunakan:

- error probability 0%;
- error probability 1%;
- error probability 5%;
- error probability 10%.

Program harus menampilkan:

1. data yang dikirim atau jumlah total bit;
2. jumlah bit error;
3. BER;
4. BER dalam persen.

Lakukan setiap kondisi minimal tiga kali.

Catat hasilnya.

Error Probability	**Percobaan**	**Bit Error**	**BER**
0%	1		
0%	2		
0%	3		
1%	1		
1%	2		
1%	3		

Error Probability	**Percobaan**	**Bit Error**	**BER**
5%	1		
5%	2		
5%	3		
10%	1		
10%	2		
10%	3		

Kemudian jelaskan:

- hubungan error probability dengan BER;
- alasan hasil tiap percobaan dapat berbeda;
- mengapa mengetahui bahwa error terjadi belum cukup jika receiver tidak memiliki mekanisme untuk mendeteksinya.

Pertanyaan terakhir menjadi pengantar ke modul berikutnya.

20. Referensi

Forouzan, B. A. (2013). *Data Communications and Networking* (5th ed.). McGraw-Hill Education.

Stallings, W. (2022). *Data and Computer Communications* (10th ed.). Pearson.

Python Software Foundation. *Python Documentation — random Module*.

NumPy Developers. *NumPy Documentation — Random Sampling and Mathematical Functions*.

Matplotlib Development Team. *Matplotlib Documentation*.